

Objects often work in hierarchies; they can use composition to instantiate other objects and can call methods of lower-level objects to ask them to do work or provide information.

To give you a clear visual representation of OOP in action, most of the examples in this book focused on widgets and other objects that can be useful in a gaming environment. I developed the `pygwidgets` and `pyghelpers` packages to demonstrate many different OOP techniques and to allow you to easily use GUI widgets in pygame programs. I hope you find these packages useful and go on to use them to develop fun or useful programs of your own.

More importantly, I hope that you recognize that object-oriented programming is a general-purpose approach that can be applied in a wide variety of circumstances. Any time you see two or more functions that need to operate on a shared set of data, you should consider building a class and instantiating an object. You may also want to consider building an object manager object to manage a group of objects.

With all that said, I'd like to offer my congratulations: you've made it to the end of the book! Although actually, this should be considered the beginning of your journey into object-oriented programming. Hopefully, the concepts described in this book have given you a framework that you can build on—but the only way to truly get a handle on how OOP works is to write lots and lots of code. Over time, you will start to notice patterns that you will use again and again in your code. Understanding how to structure your classes is a difficult process. Only through experience will it begin to become easier to ensure that you have the proper methods and instance variables in the correct classes.

Practice, practice, practice!